

src\main.cpp

```
1  /*
2  * File      : main.cpp
3  * Author    : Ding Jérémy
4  * Date      : May 2026
5  * Target    : ESP32-WROOM-32
6  *
7  * Description :
8  * Firmware for the self-balancing robot developed as part of the
9  * Bachelor's thesis "Self-Balancing Robot".
10 *
11 * Features:
12 * - Dual-loop PID controller (velocity + balance)
13 * - MPU6050 attitude estimation using a complementary filter
14 * - TMC2226 stepper motor control
15 * - Wi-Fi Access Point with WebSocket communication
16 * - Battery monitoring
17 * - Data recording for control analysis
18 */
19 #include <Arduino.h>
20
21 //----- INCLUDES -----
22 #include <WiFi.h>
23 #include <AsyncTCP.h>
24 #include <ESPAsyncWebServer.h>
25 #include <Arduino_JSON.h>
26 #include "LittleFS.h"
27 #include <SPI.h>
28 #include "TMC2226.h"
29 #include <Wire.h>
30 #include <Adafruit_MPU6050.h>
31 #include <Adafruit_Sensor.h>
32
33 //----- DEFINES -----
34
```

```
35 #define D_UART_BAUDRATE 9600
36
37 // BATTERY MANAGEMENT
38 #define D_PIN_BATTERY_LVL 35
39 #define D_BATT_RATIO 6.022 // Resistors ratio (Vbat/Vadc)
40
41 #define D_BATT_MIN_V 14.70
42 #define D_BATT_MAX_V 16.00
43
44 // ----- PID Controller Parameters -----
45 // Velocity controller
46 #define D_REG_KPV 0.62
47 #define D_REG_TDV 0.0
48 #define D_REG_TIV 0.0
49
50 // Balance controller
51 #define D_REG_KPT 3.68
52 #define D_REG_TDT 1.10
53 #define D_REG_TIT 0.0
54
55 // Rest Angle
56 #define D_BALANCE_ANGLE 2.1 // [°]
57
58 #define D_DATASEND_PERIOD 500 // Data transmission period [ms]
59 #define D_ANGLE_LIMITS {-20, 20} // Limit angles [°]
60 #define D_SPEED_CTRL_ANGLE {-3, 3} // Speed control lean angles [°]
61
62 #define D_WHEEL_RADIUS 0.055 // [m]
63 #define D_MOTOR_MAX_SPEED 5 * M_PI // [rad/s]
64
65 // WiFi credentials
66 #define D_SSID "self-balancing-bot"
67 #define D_PASSWORD "123456789"
68
69 // File management (data saving)
70 #define D_MAX_RECORDING_SIZE 2000 // (Samples)
71
```

```
72 #define D_BATTERY_SEND_PERIOD 5000 // Update battery every x [ms]
73
74 // Regulation loops frequencies
75 #define D_INNER_LOOP_FREQ 500.0
76 #define D_INNER_LOOP_PERI 1.0 / D_INNER_LOOP_FREQ
77 #define D_OUTER_LOOP_FREQ 500.0
78 #define D_OUTER_LOOP_PERI 1.0 / D_OUTER_LOOP_FREQ
79
80 // Structures
81 typedef struct PIDController
82 {
83     float kp;
84     float ki;
85     float kd;
86
87     float integral;
88     float previousError;
89
90     float outputMin;
91     float outputMax;
92 } PIDController;
93
94 typedef struct
95 {
96     float time;          // [s]
97     float theta;         // [°]
98     float theta_target; // [°]
99     float speed;         // [m/s]
100    float speed_target; // [m/s]
101 } S_DATA;
102
103 //----- FUNCTION PROTOTYPES -----
104 void init_littlefs();
105 void init_wifi();
106
107 void notify_clients(String msg);
108 void handle_websocket_message(void *arg, uint8_t *data, size_t len);
```

```
109 void on_event(AsyncWebSocket *server, AsyncWebSocketClient *client, AwsEventType type, void *arg, uint8_t *data, size_t len);
110 void init_websocket();
111
112 float PID_update(PIDController *pid, float error, float dt);
113
114 void send_angle(float s);
115 void send_target_angle(float target_angle);
116 void send_motor_state(bool state);
117
118 bool is_angle_within_range();
119 float map_f(float value, float min1, float max1, float min2, float max2);
120 float rad_s_to_ustp_s(float rad_s, uint16_t micro_stepping_value);
121 void send_recording();
122 float read_battery();
123 void send_battery(float battery_voltage);
124
125 //----- GLOBAL VARIABLES -----
126 AsyncWebServer server(80); // Web server
127 AsyncWebSocket ws("/ws"); // WebSocket endpoint
128
129 // IMU (MPU6050)
130 Adafruit_MPU6050 mpu;
131
132 // Motor drivers
133 TMC2226 tmc2226_left(0x00);
134 TMC2226 tmc2226_right(0x01);
135
136 float angleX = 0.0f;
137 float angleX_offset = 0.0f;
138
139 unsigned long lastMicros = 0;
140
141 float us_speed = 0;
142 float motor_speed = 0.0f;
143
144 float outer_loop_cnt = 0.0f;
145 float inner_loop_cnt = 0.0f;
```

```
146
147 float acceleration = 0.0f;
148
149 float speed = 0.0f;
150 float speed_target = 0.0f;
151 float rotation_speed_target = 0.0f;
152
153 float theta_target = 0.0f;
154
155 uint32_t angle_timer = 0;
156
157 float prev_gyro = 0.0f;
158
159 int8_t angle_limits[2] = D_ANGLE_LIMITS;
160 float speed_control_range[2] = D_SPEED_CTRL_ANGLE;
161
162 bool motor_state = false;
163
164 S_DATA recording_data[D_MAX_RECORDING_SIZE];
165 bool recording = false;
166 uint16_t recording_size = 0;
167
168 uint16_t micro_stepping_value = 0;
169
170 float battery_timer = 0;
171
172 // PID controllers
173 PIDController acceleration_PID =
174     {
175         .kp = D_REG_KPV,
176         .ki = D_REG_TIV,
177         .kd = D_REG_TDV,
178
179         .integral = 0.0f,
180         .previousError = 0.0f,
181
182         .outputMin = -5.0f, // max desired tilt [°]
```

```
183         .outputMax = 5.0f};
184
185 PIDController tilt_PID =
186     {
187         .kp = D_REG_KPT,
188         .ki = D_REG_TIT,
189         .kd = D_REG_TDT,
190
191         .integral = 0.0f,
192         .previousError = 0.0f,
193
194         .outputMin = -100, // Motor acceleration [rad/s^2]
195         .outputMax = 100};
196
197 //----- SETUP FUNCTION -----
198 void setup()
199 {
200     // Begin serial transmission
201     Serial.begin(D_UART_BAUDRATE); // Open serial port
202     Serial.println("Serial1 initialized");
203
204     // Initializes TMC
205     tmc2226_left.init();
206     tmc2226_right.init();
207
208     // Set chopper config
209     TMC2226::CHOPCONF chopconf;
210
211     chopconf.value = D_TMC_REGDFV_CHOPCONF; // DEFAULT VALUE
212     chopconf.bits.mres = 0b0001;           // Microstepping
213     micro_stepping_value = 128;
214
215     tmc2226_left.write_to_register(TMC2226::E_REG_CHOPCONF, chopconf.bytes);
216     tmc2226_right.write_to_register(TMC2226::E_REG_CHOPCONF, chopconf.bytes);
217
218     // Start MPU
219     if (!mpu.begin())
```

```
220 {
221     Serial.println("MPU6050 not found");
222     while (1)
223         ;
224 }
225
226 // Initialize MPU settings
227 mpu.setAccelerometerRange(MPU6050_RANGE_8_G);
228 mpu.setGyroRange(MPU6050_RANGE_500_DEG);
229 mpu.setFilterBandwidth(MPU6050_BAND_21_HZ);
230
231 // Initialize battery pin
232 pinMode(D_PIN_BATTERY_LVL, INPUT);
233
234 // Start WIFI and server
235 init_wifi();
236 init_littlefs();
237 init_websocket();
238
239 // Serve root HTML
240 server.on("/", HTTP_GET, [](AsyncWebServerRequest *request)
241     { request->send(LittleFS, "/index.html", "text/html"); });
242
243 // Serve static files
244 server.serveStatic("/", LittleFS, "/");
245
246 server.begin();
247
248 delay(50);
249 // Check if the bot is in a correct position
250 // If angle out of safe range.
251 if (is_angle_within_range())
252 {
253     motor_state = true;
254     tmc2226_left.enable();
255     tmc2226_right.enable();
256     send_motor_state(true);
```

```
257     }
258     else
259     {
260         send_motor_state(false);
261     }
262 }
263
264 //----- MAIN LOOP -----
265 void loop()
266 {
267     // Time reading
268     unsigned long now = micros();
269     float dt = (now - lastMicros) * 1e-6f;
270     lastMicros = now;
271
272     // OUTER LOOP
273     outer_loop_cnt += dt;
274     if (outer_loop_cnt > D_OUTER_LOOP_PERI)
275     {
276         // Update target based on joystick input (position_control)
277         float speed_error = -(speed_target - speed);
278         // Generate angle theta based on position Error
279         theta_target = PID_update(&acceleration_PID, speed_error, outer_loop_cnt);
280         outer_loop_cnt = 0;
281     }
282
283     // INNER LOOP
284     inner_loop_cnt += dt;
285
286     if (inner_loop_cnt > D_INNER_LOOP_PERI)
287     {
288         // theta_target = speed_target;
289         // ANGLE READING
290         sensors_event_t accel, gyro, temp;
291         mpu.getEvent(&accel, &gyro, &temp);
292
293         // Accelerometer angle
```



```
294     float accelAngleX =
295         atan2(accel.acceleration.x,
296             sqrt(accel.acceleration.y * accel.acceleration.y +
297                 accel.acceleration.z * accel.acceleration.z)) *
298         180.0f / PI;
299
300     // Gyro rate (Adafruit returns rad/s)
301     float gyroRateX = gyro.gyro.x * 180.0f / PI;
302     gyroRateX = /*0.5f * prev_gyro + 0.5f * */ gyroRateX;
303
304     // Complementary filter
305     angleX = 0.9f * (angleX + gyroRateX * inner_loop_cnt) + 0.1f * accelAngleX;
306     prev_gyro = gyroRateX;
307     // Defines the error from speed (and saved offset + Hard coded offset)
308     float thetaError = D_BALANCE_ANGLE + theta_target + angleX_offset - angleX;
309
310     // Generate motor speed from inner loop
311     float acceleration =
312         PID_update(&tilt_PID, thetaError, inner_loop_cnt);
313
314     // Convert rad/s to ustp/stp
315     motor_speed += acceleration * inner_loop_cnt;
316     motor_speed = motor_speed > D_MOTOR_MAX_SPEED ? D_MOTOR_MAX_SPEED : motor_speed < -D_MOTOR_MAX_SPEED ? -D_MOTOR_MAX_SPEED
317                                                         : motor_speed;
318
319     us_speed = rad_s_to_ustp_s(motor_speed, micro_stepping_value);
320     float motor_speed_delta = rad_s_to_ustp_s(rotation_speed_target, micro_stepping_value);
321
322     // Chariot speed
323     speed = motor_speed * D_WHEEL_RADIUS;
324
325     // run motors (standalone speed)
326     tmc2226_left.run_speed((int32_t)us_speed + motor_speed_delta);
327     tmc2226_right.run_speed(-(int32_t)us_speed + motor_speed_delta);
328
329     // If currently recording
330     if (recording && recording_size < D_MAX_RECORDING_SIZE)
```

```
331     {
332         recording_data[recording_size].time = millis();
333         recording_data[recording_size].speed_target = speed_target;
334         recording_data[recording_size].speed = speed;
335         recording_data[recording_size].theta_target = theta_target;
336         recording_data[recording_size].theta = angleX;
337
338         recording_size++;
339     }
340     inner_loop_cnt = 0;
341 }
342
343 // If angle out of safe range.
344 if (!is_angle_within_range())
345 {
346
347     if (motor_state)
348     {
349         motor_state = false;
350         tmc2226_left.disable();
351         tmc2226_right.disable();
352         send_motor_state(false);
353     }
354 }
355 if (!motor_state)
356 {
357     motor_speed = 0;
358 }
359
360 // Send angle and speed to clients (if not recording)
361 if (!recording)
362 {
363     if (millis() - angle_timer > D_DATASEND_PERIOD)
364     {
365         angle_timer = millis();
366         send_target_angle(theta_target);
367         send_angle(angleX);
368     }
369 }
```

```
368     }
369     if (millis() - battery_timer > D_BATTERY_SEND_PERIOD)
370     {
371         battery_timer = millis();
372         send_battery(read_battery());
373     }
374 }
375 }
376
377 //----- FUNCTION IMPLEMENTATIONS -----
378
379 /// @brief Transforms rad/s to microstep/s
380 /// @param rad_s
381 /// @param micro_stepping_value
382 /// @return
383 float rad_s_to_ustp_s(float rad_s, uint16_t micro_stepping_value)
384 {
385     const float steps_per_rev = 200.0f; // typical stepper motor (1.8°)
386
387     float rev_per_sec = rad_s / (2.0f * M_PI);
388     float steps_per_sec = rev_per_sec * steps_per_rev;
389
390     return steps_per_sec * micro_stepping_value;
391 }
392 /// @brief Reads current angleX and assess if in the range
393 /// @return
394 bool is_angle_within_range()
395 {
396     return (angleX > angle_limits[0] && angleX < angle_limits[1]);
397 }
398 /// @brief Send the state of the motor to the clients
399 /// @param state
400 void send_motor_state(bool state)
401 {
402     JSONVar json;
403     if (state)
404         json["id"] = "motors_on";
```

```
405     else
406         json["id"] = "motors_off";
407     String msg = JSON.stringify(json);
408     notify_clients(msg);
409 }
410 /// @brief Sends the current angle to the clients
411 /// @param theta
412 void send_angle(float theta)
413 {
414     // Create JSON message, send it
415     JSONVar json;
416     json["id"] = "angle";
417     json["value"] = theta;
418     String msg = JSON.stringify(json);
419     notify_clients(msg);
420 }
421 /// @brief Send entire recording to client
422 void send_recording()
423 {
424     // Set timestamp to 0 (for the first one)
425     float first_time = recording_data[0].time;
426     // Send the recording transmit signal
427     ws.textAll("{\"id\":\"save_start\"}");
428     ws.textAll("Time [ms],Speed_target [m/s],Speed [m/s],Theta_Target [°], Theta [°]");
429     String chunk;
430
431     for (int i = 0; i < recording_size; i++)
432     {
433         chunk += String(recording_data[i].time - first_time, 3) + "," +
434             String(recording_data[i].speed_target, 3) + "," +
435             String(recording_data[i].speed, 3) + "," +
436             String(recording_data[i].theta_target, 3) + "," +
437             String(recording_data[i].theta, 3) + "\n";
438
439         // Send data chunk wise
440         if (chunk.length() > 4096)
441         {
```

```
442     ws.textAll(chunk);
443
444     chunk = "";
445 }
446 }
447
448 if (chunk.length())
449 {
450     ws.textAll(chunk);
451 }
452
453 ws.textAll("save_end");
454 }
455
456 /// @brief Sends battery infos to clients
457 /// @param battery_voltage
458 void send_battery(float battery_voltage)
459 {
460     // Create JSON message, send it
461     JSONVar json;
462     json["id"] = "battery";
463     json["voltage"] = String(battery_voltage, 2);
464     json["percentage"] = String(map_f(battery_voltage, D_BATT_MIN_V, D_BATT_MAX_V, 0, 100), 2);
465     String msg = JSON.stringify(json);
466     notify_clients(msg);
467 }
468
469 /// @brief Sends target angle to clients
470 /// @param speed
471 void send_target_angle(float target_angle)
472 {
473     // Create JSON message, send it
474     JSONVar json;
475     json["id"] = "target_angle";
476     json["value"] = target_angle;
477     String msg = JSON.stringify(json);
478     notify_clients(msg);
```

```
479 }
480 // PID //
481
482 /// Update PID based on a given discrete jump.
483 float PID_update(PIDController *pid, float error, float dt)
484 {
485     // Derivative
486     float derivative = (error - pid->previousError) / dt;
487
488     // Compute output WITHOUT updating integral yet
489     float output =
490         pid->kp * error +
491         pid->ki * pid->integral +
492         pid->kd * derivative;
493
494     // Check if output would saturate
495     bool saturatedHigh = output > pid->outputMax;
496     bool saturatedLow = output < pid->outputMin;
497
498     // Anti-windup: only integrate if it helps leave saturation
499     if (!(saturatedHigh && error > 0) &&
500         !(saturatedLow && error < 0))
501     {
502         pid->integral += error * dt;
503     }
504
505     // Recompute output with updated integral
506     output =
507         pid->kp * error +
508         pid->ki * pid->integral +
509         pid->kd * derivative;
510
511     // Saturate output
512     if (output > pid->outputMax)
513         output = pid->outputMax;
514     else if (output < pid->outputMin)
515         output = pid->outputMin;
```

```
516
517     pid->previousError = error;
518
519     return output;
520 }
521 // SERVER //
522 /*
523  * Initializes the LittleFS file system
524  */
525 void init_littlefs()
526 {
527     if (!LittleFS.begin(true))
528     {
529         Serial.println("An error has occurred while mounting LittleFS");
530     }
531     else
532     {
533         Serial.println("LittleFS mounted successfully");
534     }
535 }
536
537 /*
538  * Initializes the ESP32 in Access Point mode
539  */
540 void init_wifi()
541 {
542     WiFi.mode(WIFI_AP);
543     WiFi.softAP(D_SSID, D_PASSWORD);
544     Serial.println("Starting Access Point...");
545     Serial.print("IP Address: ");
546     Serial.println(WiFi.softAPIP());
547 }
548
549 /*
550  * Sends a text message to all connected WebSocket clients
551  */
552 void notify_clients(String msg)
```

```
553 {
554     ws.textAll(msg);
555 }
556
557 /*
558  * Handles incoming WebSocket messages from the client
559  */
560 void handle_websocket_message(void *arg, uint8_t *data, size_t len)
561 {
562     // Get frame infos
563     AwsFrameInfo *info = (AwsFrameInfo *)arg;
564
565     // Check if the received websocket message is valid
566     if (info->final && info->index == 0 && info->len == len && info->opcode == WS_TEXT)
567     {
568         // Parse Data and Message.
569         String message = (char *)data;
570         JSONVar msg = JSON.parse(message);
571
572         // If the received message is correct
573         if (JSON.typeof(msg) == "object")
574         {
575             String msg_id = (const char *)msg["id"];
576             if (msg_id == "motors_off")
577             {
578                 motor_state = false;
579                 tmc2226_left.disable();
580                 tmc2226_right.disable();
581                 send_motor_state(false);
582             }
583             else if (msg_id == "motors_on")
584             {
585                 if (is_angle_withing_range())
586                 {
587                     tmc2226_left.enable();
588                     tmc2226_right.enable();
589                     motor_state = true;
```



```
590         send_motor_state(true);
591     }
592 }
593 else if (msg_id == "calibrate")
594 {
595     // Save angle offset
596     angleX_offset = angleX;
597
598     // Transmit saved angle to server
599     JSONVar json;
600     json["id"] = "calibration_angle";
601     json["value"] = angleX_offset;
602     String msg = JSON.stringify(json);
603     notify_clients(msg);
604 }
605 else if (msg_id == "pid_values")
606 {
607     // Safety check
608     if (msg["balance"].hasOwnProperty("kp"))
609     {
610         tilt_PID.kp = (double)msg["balance"]["kp"];
611         tilt_PID.ki = (double)msg["balance"]["ti"];
612         tilt_PID.kd = (double)msg["balance"]["td"];
613     }
614
615     if (msg["velocity"].hasOwnProperty("kp"))
616     {
617         acceleration_PID.kp = (double)msg["velocity"]["kp"];
618         acceleration_PID.ki = (double)msg["velocity"]["ti"];
619         acceleration_PID.kd = (double)msg["velocity"]["td"];
620     }
621 }
622 else if (msg_id == "pid_request")
623 {
624     JSONVar json;
625     json["id"] = "pid_values";
626     json["balance"]["kp"] = tilt_PID.kp;
```

```
627     json["balance"]["td"] = tilt_PID.kd;
628     json["balance"]["ti"] = tilt_PID.ki;
629     json["velocity"]["kp"] = acceleration_PID.kp;
630     json["velocity"]["td"] = acceleration_PID.kd;
631     json["velocity"]["ti"] = acceleration_PID.ki;
632     String msg = JSON.stringify(json);
633     notify_clients(msg);
634 }
635 else if (msg_id == "joystick-left")
636 {
637     // Adjust speed target from joystick
638     speed_target = map_f((double)msg["y"], -100, 100, speed_control_range[0], speed_control_range[1]);
639 }
640 else if (msg_id == "joystick-right")
641 {
642     rotation_speed_target = map_f((double)msg["x"], -100, 100, speed_control_range[0], speed_control_range[1]);
643 }
644 else if (msg_id == "record_start")
645 {
646     recording_size = 0;
647     recording = true;
648 }
649 else if (msg_id == "record_stop")
650 {
651     send_recording();
652     recording = false;
653 }
654 }
655 }
656 }
657
658 /*
659  * WebSocket event callback handler
660  */
661 void on_event(AsyncWebSocket *server, AsyncWebSocketClient *client, AwsEventType type, void *arg, uint8_t *data, size_t len)
662 {
663     switch (type)
```

```
664 {
665     case WS_EVT_CONNECT:
666         Serial.printf("WebSocket client #%u connected from %s\n", client->id(), client->remoteIP().toString().c_str());
667         break;
668     case WS_EVT_DISCONNECT:
669         Serial.printf("WebSocket client #%u disconnected\n", client->id());
670         break;
671     case WS_EVT_DATA:
672         handle_websocket_message(arg, data, len);
673         break;
674     case WS_EVT_PONG:
675     case WS_EVT_ERROR:
676         break;
677 }
678 }
679
680 /*
681  * Initializes the WebSocket server and binds events
682  */
683 void init_websocket()
684 {
685     ws.onEvent(on_event);
686     server.addHandler(&ws);
687 }
688
689 /// @brief Maps a value from one range to another and clamps the output.
690 /// @param value Input value
691 /// @param min1 Input range minimum
692 /// @param max1 Input range maximum
693 /// @param min2 Output range minimum
694 /// @param max2 Output range maximum
695 /// @return Mapped and clamped value
696 float map_f(float value, float min1, float max1, float min2, float max2)
697 {
698     if (max1 == min1)
699     {
700         // Avoid division by zero
```

```
701     return min2;
702 }
703
704 float t = (value - min1) / (max1 - min1);
705 float result = min2 + t * (max2 - min2);
706
707 // Clamp output
708 if (min2 < max2)
709 {
710     if (result < min2)
711         result = min2;
712     if (result > max2)
713         result = max2;
714 }
715 else
716 {
717     // Handle reversed output ranges
718     if (result < max2)
719         result = max2;
720     if (result > min2)
721         result = min2;
722 }
723
724 return result;
725 }
726
727 /// @brief Read battery level in mV
728 /// @return
729 float read_battery()
730 {
731     uint32_t voltage = analogReadMilliVolts(D_PIN_BATTERY_LVL);
732     float battery_voltage = voltage * D_BATT_RATIO / 1000; // [V]
733     return battery_voltage;
734 }
735
```